

Technical Portfolio

Payment Architecture & Platform Design

Tony

Engineering Leader | Payment Architect | Crypto Platform

Senior Engineering Manager, Crypto Platform & Infrastructure

Singapore | Ex YouTrip | Ex Aspire | Ex Thunes

This portfolio summarizes the current platform design developed for a next generation on/off ramp and multi currency account system: one Core service (money truth), one Payout Service (rails), single source code, multi region by design, Singapore first.

It is aligned with the living design canon: BLUEPRINT, non negotiable implementation principles, multi region compliance, and launch architecture.

Updated 2026 | Architecture & product design focus

1. Background

I design and lead payment platforms that move money safely at scale: double entry ledgers, multi currency accounts (MCA), FX, card rails, payouts, and crypto/stablecoin infrastructure. Career focus has been regulated fintech: systems that banks, PSPs, and merchants depend on, plus the teams that operate them.

Selected experience

- * Crypto platform & infrastructure engineering leadership for stablecoin payment rails.
- * YouTrip: led 20+ engineers (MCA / YouBiz); Family Card, 3DS, multi currency wallets and cards at large scale; engineering career ladder.
- * Aspire: payment architecture for multi currency business accounts across Singapore, Hong Kong, and Australia; VA, payout, FX providers.
- * Thunes: tech lead for GrabPay and TikTok pay corridors; high volume APAC; 15+ partners including Alipay, DBS, RippleNet, MoneyGram.

Domain depth

Ledger design, multi currency wallets, cross border payouts, card issuing, on ramp / off ramp, webhooks, reconciliation, AML aware flows, multi region expansion

2. Non negotiable design principles

These principles are mandatory during implementation. They are not aspirational later cleanups.

P0 Single source code, multi region by design

- * One source tree, one migration chain, one binary per service for all markets.
- * Never fork the codebase by region (no core eu / core us product forks).
- * Launch Singapore (home_region = sg). Expand to US, EU, and beyond by enabling regions: config, capacity, licenses: not a platform rebuild.
- * home_region on all customer scoped data from day one; REGIONAL BY ROW ready; per region house books; no hard coded geography in domain logic.
- * North star: write the platform once; pin data by region; turn markets on with config and capacity.

P1 Money truth

- * Core owns all financial state. No other service writes balances or postings.
- * PostJournalEntry is the only ledger write path.
- * Decimal money only (never float). Append only history; reversals for corrections.
- * Balanced journals; idempotent (source, external_ref); available vs posted balances.

P2 Modular monolith at launch

- * Modules = Go packages with exclusive table ownership and interfaces.
- * One CockroachDB database for Core modules at launch.
- * Outbox in the same DB transaction as state change; Core owns the money saga.

P3 Product model

- * Crypto and fiat are first class on the same MCA and ledger.
- * Profiles: party_type business or person. Internal parties only.
- * Recipients/payers are external counterparties: never full profiles.
- * Public API is merchant shaped: /payments, /payouts, /transfers.

3. Platform architecture (launch)

Two backend systems. Core is a modular monolith. Payout Service is the rails pipe. Clients hit Core API; Core dispatches to Payout; external banks, custody, and AML vendors sit behind those boundaries.

Service and data ownership

```

Clients:  Web / Checkout, Mobile, Merchant API
      |
Core API (auth, rate limits, REST)
      |
Core Service ..... CockroachDB (+ Redis)
      Transaction processing, Ledger, MCA, Onboarding
      |
Payout Service ..... MySQL only
      Bank, Crypto, FX integration
      |
External: Banks/PSPs, Crypto custody, AML vendors
  
```

Core modules (same deployable)

API, Txn engine, Ledger (PostJournalEntry), Account / MCA, Onboarding (business or person, home_region), Compliance client, Quote / fees, Webhook, Outbox worker. Packages talk via interfaces, not each other's tables.

Why this shape

- * Solo / small team operable without microservice sprawl.
- * Extraction triggers (account, onboarding, AML, Kafka) are named: not date driven.
- * Payout Service reuses existing multi provider rails; crypto is new adapters.
- * No DynamoDB for payouts; no required separate gateway or AML service at launch.

4. On ramp and off ramp (one engine)

Both directions share one transaction engine, one ledger, one MCA model (fiat + crypto), one home_region tree, and one payout service adapter model.

On ramp (collection / pay in)

- * API: POST /v1/payments (hosted or API mode).
- * Ledger: pending credit, then post after AML clear and chain/fiat finality.
- * AML: async with hold (funds pending, not spendable until clear).
- * Rails: deposit address / VA / pay in webhooks via Payout Service.
- * M1 ship: in kind crypto (e.g. USDC in to USDC balance), no FX required.
- * M3+: local currency settlement with rate lock and fx_position.

Off ramp (payout)

- * API: POST /v1/payouts then POST .../confirm within quote TTL.
- * Ledger: debit hold first (available down); post on settle or void on fail/expiry.
- * AML: sync, fail closed before dispatch.
- * Rails: dispatch withdrawal via Payout Service (bank or crypto wallet).
- * Statuses include awaiting_confirmation, processing, completed, failed, expired.

Internal transfer

- * Account to account when both sides are internal: ledger only, no Payout Service.
- * Cross region internal transfer forbidden in v1 (same home_region only).

5. Multi region compliance (Singapore first)

Deploy may start as a single region. The data model and request path must already be multi region correct so expansion does not rebuild infrastructure.

- * home_region set at profile creation (country/license map); inherited by accounts, ledger, transactions.
- * Customer tables: REGIONAL BY ROW ready. Reference data (e.g. currencies): GLOBAL.
- * House accounts (omnibus, fee, FX, suspense, aml_hold) provisioned per region.
- * PostJournalEntry rejects mixed region posting lines.
- * PII documents in per region object storage; not in SQL.
- * Rollout playbook: enable region in registry, seed house books, buckets/Redis, same image, corridors.

6. Product pillars (one core)

- * Merchant rails (launch): crypto collection and payouts, then fiat legs: merchant shaped API over owned rails.

- * Business multi currency accounts (SME surfaces).

- * Consumer multi currency accounts (personal surfaces).

All three share the same ledger, party model (business or person), MCA (fiat + crypto), transaction engine, and rails. Schema supports both party types from day one; business KYB ships first.

Public API surface (merchant)

- * POST/GET /v1/payments: collections

- * POST /v1/payouts + confirm: payouts

- * POST /v1/transfers: internal

- * Balances, transactions, recipients, rates, signed webhooks

- * Idempotency Key; amounts as decimal strings; API keys at launch

7. Build milestones (sellable increments)

- * M0 Foundation: ledger holds/post/void, home_region, MCA, API keys, outbox, region registry (sg).

- * M1 Crypto collection MVP: custody adapter, deposit flow, payments API in kind, webhooks, recon.

- * M2 Crypto payouts + sandbox: create/confirm/dispatch, AML, checkout, public sandbox.

- * M3 Fiat legs: local settlement, rate lock, fiat payouts, fx_position visibility.

- * M4 Polish: transfers, short/over paid, recipient email wallet capture, ops tooling.

8. Stack and integrations

Primary stack

Go, CockroachDB (Core), MySQL (Payout Service), Redis, gRPC, Docker / Kubernetes, AWS, Datadog / Prometheus / Grafana, Kafka where already used on rails systems

Banks (sample)

J.P. Morgan, DBS, OCBC, Standard Chartered, Citi, HDFC, SeaBank, Maybank, KBank, Siam Commercial Bank, CZBank, Bank Alfalah, 9Pay, and other regional banks

Mobile wallets & networks

Alipay, WeChat Pay, GrabPay, PayPal, Stripe, MoneyGram, RippleNet

FX & cross border

Wise, CurrencyCloud, Thunes

Custody & crypto infrastructure

Fireblocks, BitGo, Copper, Circle, Coinbase Prime, QuickNode

9. Selected prior systems

Payout service

Multi currency global payouts microservice; 40+ providers (JPM, DBS, Wise, HDFC, PayPal, Stripe, RippleNet, and more). Event driven, high throughput, bank facing reliability.

Payment acceptance & webhooks

Real time provider acknowledgement path; fraud workflows under MAS/HKMA expectations; horizontal scale with AWS, Kafka, Redis, Datadog.

High volume processors

Grab and TikTok payment integrations supporting millions of daily transactions; routing and settlement tuned for regional rails.

Contact

LinkedIn: [linkedin.com/in/tony007](https://www.linkedin.com/in/tony007)

GitHub: github.com/Aibier

Email: tony.aizize@you.co

Design sources: [BLUEPRINT](#), [docs/NON-NEGOTIABLE.md](#), [docs/multi-region-compliance.md](#), [docs/architecture-launch.md](#), [taymas-platform-design.md](#) (core-service).